Open in Colab

Lab 9.1 Machine Learning 2 - Machine Learning Implementation to Edge Device

Understanding and Analyzing Data

Learning Goals

Students will be able to:

1. Implement a data collection process for TinyML
2. Deploy a TinyML model using TensorFlow
3. Build a monitoring system using TinyML

1.1 Introduction

In this lab, we will implement a machine learning model to Raspberry Pi as an application of IIoT smart monitoring system to predict running conditions of the axial flow fan (AFF). This lab is broken down into three main sections: **1) Understanding and analyzing data, 2) Machine learning implementation to Raspberry Pi**, and **3) Building up the entire monitoring system.** The entire monitoring system and the data pipeline are illustrated in Figure 1.

Note that the sample code for this lab is on Github repo.

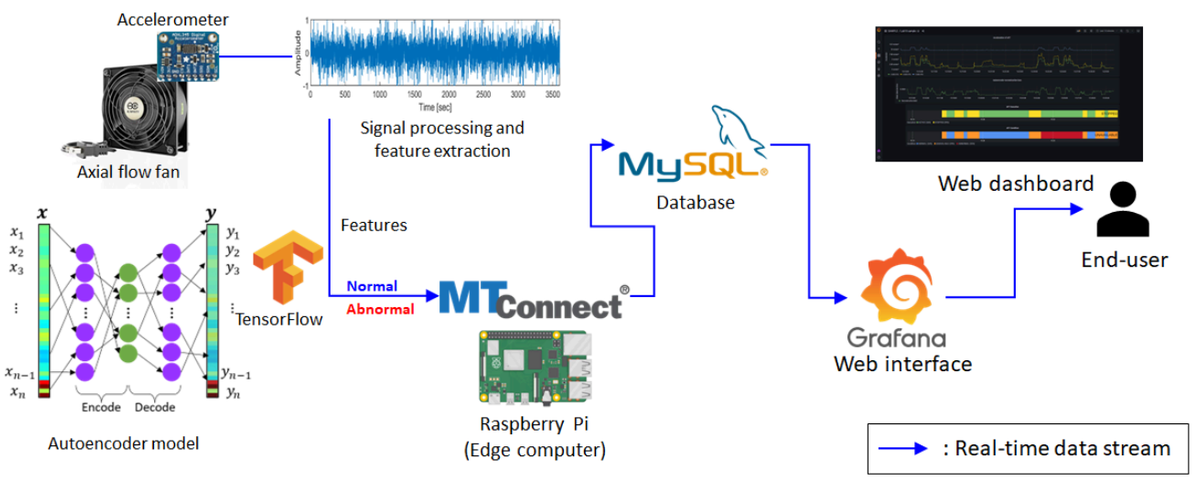


Figure 1 Schematic of entire monitoring system and data pipeline

1.2 Acceleration of the AFF

In this section, we will go over the algorithm of the smart monitoring system and determine some variables for the smart monitoring system. The goal of the monitoring system for the AFF is to visualize in real-time 1) the vibration of the AFF, 2) the execution ('ACTIVE' or 'STOPPED'), and 3) the prediction of running condition ('NORMAL' or 'ABNORMAL'). Obviously, the prediction of running conditions based on the machine learning model should be only done when the AFF is in an 'ACTIVE' state. If the AFF is not running, the prediction of anomaly does not mean anything.

The algorithm and the flowchart of the AFF monitoring system is illustrated in Figure 2. This data flow will be repeated in a loop in your program. In Lab8, we developed an anomaly detection model (autoencoder) for the second decision ("Is AFF normal?"). But we do not have a model for the first decision ("Is AFF running?"). Of course, a model to determine the running state ('ACTIVE' or 'STOPPED') can be developed by using machine learning approach as we performed in Lab8. However, let's make the simplest rule based on the measured vibration data. The idea is that when the AFF is running, the acceleration of each axis of the sensor must increase. By a simple logic such as a certain axis acceleration is lower than a specific acceleration value, a threshold on running, we are able to determine if the AFF is running or not. In other words, if the acceleration of a certain axis is higher than the threshold value, we can say the AFF is running, e.i., 'Execution' of AFF is 'ACTIVE'.

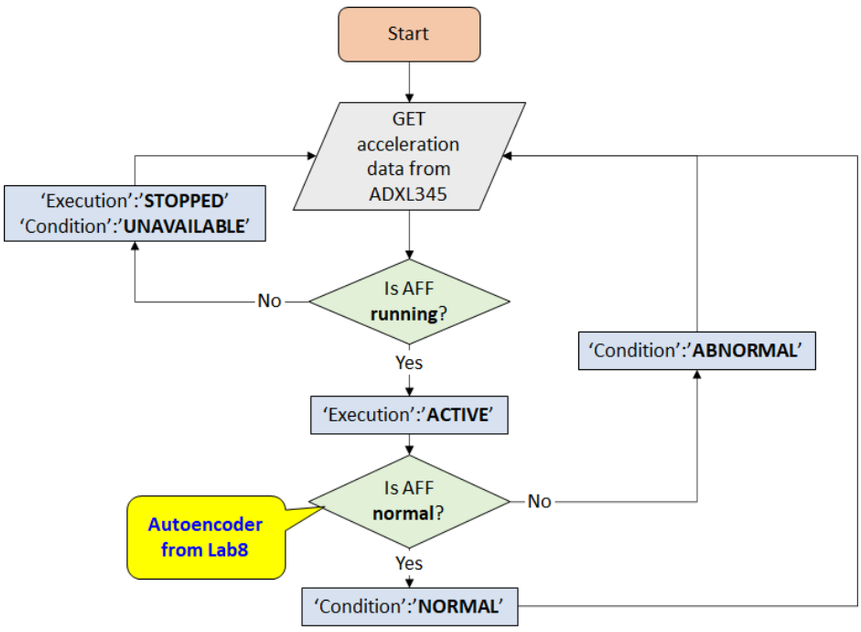


Figure 2 Flow chart of AFF monitoring system

The sample plots according to the various conditions of the AFF are in Lab9 APPENDIX at the end of this manual. On the time domain plots, you see the RMS values according to the axis. As you see, when the machine is stopped (Figure A. 2), the RMS (root mean square) accelerations are the minimum on the x- and y-axis. In this example, the sample data shows that **it is reasonable to set the execution RMS threshold of the x-axis as 1 m/s^2.** However, the RMS threshold value may be different according to the AFF, and the sensor configurations.

First, **deploy ADXL345 sensor to the AFF as the same to Lab8**. To determine the RMS threshold for the execution of the AFF, perform Task 1 below. If you are not sure about ADXL sensor deployment, plese review Lab2.1 Manual.

Task 1.1

Run lab9_sample1.py on Raspberry Pi to check the RMS values on each axis according to the executions ('STOPPED' and 'ACTIVE') of the AFF. In the AFF is the 'ACTIVE' condition, try various rotational speeds.

Determine the RMS threshold value of a specific axis. What are the RMS value and the axis to determine if the AFF is running?

Axis	STOPPED RMS Range (m/s^2)	RUNNING RMS Range (m/s^2)	Separation	Selected
X	0.907 – 1.009	1.823 – 2.128	Very High	Yes
Y	0.832 – 0.892	1.201 – 2.191	Moderate	No
Z	9.940 – 10.268	10.076 – 10.257	Very Low	No

The X-axis was selected because it shows the clearest separation between STOPPED and RUNNING states.

RMS Threshold Calculation

The RMS threshold is calculated as the midpoint between the maximum STOPPED RMS and minimum RUNNING RMS values:

$$\text{Threshold} = \frac{\text{RMS}_{\text{stopped,max}} + \text{RMS}_{\text{running,min}}}{2}$$

Substituting the values:

$$\text{Threshold} = \frac{1.009 + 1.823}{2} \approx 1.4\,\text{m/s}^2$$

The RMS threshold was determined as 1.4 m/s^2 using the midpoint between the maximum STOPPED RMS and minimum RUNNING RMS values.

Task 1.2

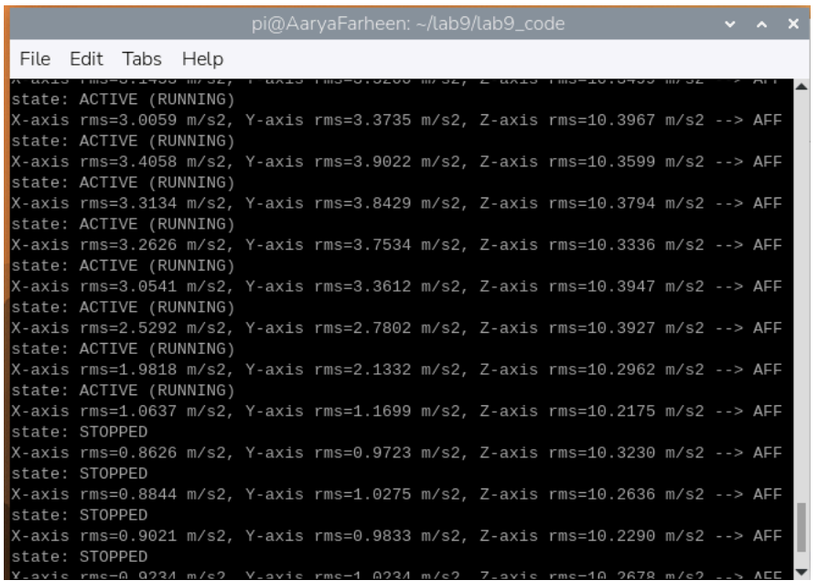
Modify lab9_sample1.py so that you can see the execution (running state) of the AFF.

a. Use 'if' and 'else' statement.

b. The example of the result is shown in Figure 3.

c. By turning the AFF on and off repeatedly, confirm if your logic and the rms threshold is effective.

d. Attach the capture of Terminal or Thonny Shell to the report.



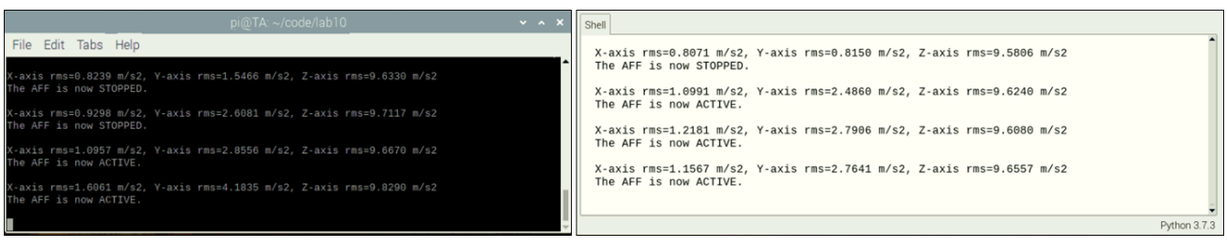


Figure 3 Check the rms values and the execution of AFF

1.3 Finding Minimum and Maximum Value of your Training Data Set

In Lab9, you developed your own autoencoder model for the anomaly detection of the AFF. When training and validating the model, we used the normalized data set. Other than the threshold (MAE, mean absolute error, loss) for the autoencoder model, when you implement the model to Raspberry Pi, you need to use the minimum and the maximum to normalize the extracted features in order to employ your model. The data flow in real time to predict running conditions of the AFF is illustrated in Figure 4. To normalize the input feature, we should know the minimum and the maximum values when training. Likewise, for the anomaly detection based on the autoencoder model, we should know the MAE loss threshold.

To get the minimum and maximum values from the collected data, perform TASK 2. The lab9_sample2.py is prepared based on the training model in Lab8 to check the minimum and maximum values of the input features.

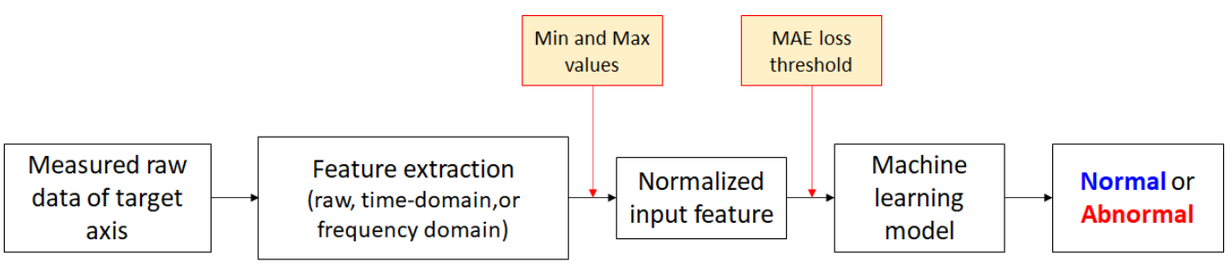


Figure 4 Real-time data flow for machine learning model to determine running conditions of AFF

To check the minimum and maxium values of your input feature of the training model, follow the steps below.

1. Run lab9_sample2.py on laptop or on the code blocks below after loading your data to check the minimum and the maximum values of the extracted input feature of the model.

   a. The collected datasets must be in the same directory as the Python script if you run the code on your laptop.

   b. Upload the collected datasets on this Colab session if you run the code below.

2. The script is incomplete as is, so you need to complete the script.

   a. Variables you need to complete:

   - normal_data_file (line 44)
   - abnormal_data_file (line 45)
   - AXIS (line 58)
   - input_feature (line 80)

```
#This code allows you to upload a file from your local drive
from google.colab import files
uploaded = files.upload()

# The below code block is lab9_sample2.py
```

Choose Files No file chosen Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

```
Saving normal.csv to normal.csv
Saving abnormal.csv to abnormal.csv
```

```
import numpy as np

import pandas as pd

import tensorflow as tf

from tensorflow import keras

from scipy import stats, fft


# Time-domain data

def timeFeatures(data):

    feature = [] # initialize feature list

    for i in range(len(data)):

        mean = np.mean(data[i]) # mean

        std = np.std(data[i]) # standard deviation

        rms = np.sqrt(np.mean(data[i] ** 2)) # root mean squre

        peak = np.max(abs(data[i])) # peak

        skew = stats.skew(data[i]) # skewness

        kurt = stats.kurtosis(data[i]) # kurtosis

        cf = peak/rms # crest factor

        # number of feature of each measurement = 7

        feature.append(np.array([mean,std,rms,peak,skew,kurt,cf], dtype=float))

    feature = np.array(feature)

    return feature # feature list, each element is numpy array with datatype float


# DFT magnitude data

def freqFeatures(data):

    feature = []

    for i in range(len(data)):

        N = len(data[i]) # number of data

        yf = 2/N*np.abs(fft.fft(data[i])[:N//2]) # yf is DFT signal magnitude

        yf[0] = 0

        feature.append(np.array(yf))

    feature = np.array(feature)

    return feature


def tensorNormalization(data): # data as numpy array

    min_val = tf.reduce_min(data) # get min val

    max_val = tf.reduce_max(data) # get max val

    data_normal = (data - min_val) / (max_val - min_val) # get normalized data as numpy array

    minVal = min_val.numpy() # convert minimum tensor to numpy

    maxVal = max_val.numpy() # convert maximum tensor to numpy

    return tf.cast(data_normal, tf.float32), minVal, maxVal # tensorarray, float 32 datatype, min, max


## data loading

# All files should be in the same directory (folder)

normal_data_file = "normal.csv" # normal condition filename: You must change this!

abnormal_data_file = "abnormal.csv" # abnormal condition filename: You must chage this!


df_normal = pd.read_csv(normal_data_file) # normal dataframe

df_abnormal = pd.read_csv(abnormal_data_file) # abnormal dataframe


frames = [df_normal, df_abnormal] # frame list to merge two dataframes into one


df = pd.concat(frames) # new concatenated dataframe
```

```
## Data Transformation

# X-axis: 'Xacc array [m/s2]'

# Y-axis: 'Yacc array [m/s2]'

# Z-axis: 'Zacc array [m/s2]'

AXIS = 'Xacc array [m/s2]' # Select one (for your model) of above axes


# Exploding the values contained in selected column and converting the string values into float values

df_new = pd.concat([df['Condition'],df[AXIS].str.split(' ', expand=True).astype(float)], axis=1) # transform space delimited array to each valu

ds = df_new.copy() # make ds by copying df


#Converting the Classifier into binary values

ds.loc[df['Condition'] == 'Normal', 'Status'] = 1 # if Condition column is 'Normal', Give 'Status' Column 1

ds.loc[df['Condition'] == 'Abnormal', 'Status'] = 0 # if Condition column is 'Abnormal', Give 'Status' Column 0

ds.drop('Condition', axis=1, inplace=True) # drop 'Condition' column (the first column)


data = ds.values

# Define Raw data W/O signal processing

raw_data = data[:,:-1]

# Labels: The last column

labels = data[:,-1]


time_data = timeFeatures(raw_data) # define time domain feature

freq_data = freqFeatures(raw_data) # define frequency domain feature (DFT)


## Data (feature) selection and Split training and validation dataset

# Feature selection

input_feature = raw_data # time_data, or freq_data


## finally print out the min value and the max values of the input feature

print("The minimum is {} and the maximum is {}.".format(tensorNormalization(input_feature)[1], tensorNormalization(input_feature)[2]))
```

```
The minimum is -9.649743599999999 and the maximum is 8.0806796.
```

Task 1.3 Answer the questions below regarding your machine learning feature and threshold.

1. What is the input feature? (Axis and type of feture among raw, time feature, frequency feature)

The selected input feature is the raw acceleration data from the X-axis. No additional signal processing such as time-domain or frequency-domain feature extraction was applied.

---

2. What is the minimum value of the input feature?

Input Feature Minimum Value

- **Minimum:** -9.6497 m/s²